SW Engineering CSC648/848 Section 6

Nautilus

AI-assisted PRD drafting

Milestone 3

Project Status and UI Review

Team 5

Date: 11/03/2026

Team Member	GitHub Username	Roles
Aksh Patel	AkshPatel63	Front-End Lead, GitHub Co-Lead
Giovannie Silva	Gsilva-code	AI Master
Leman Yuksel	LemanYuksel	Scrum Master
Marco Garcia	MarcoGarcia650	Back-End Lead
Om Pandya	Atom3798	Team Lead, AI Master
Philip Chen	philipchensfsu	GitHub Master
Vineela Vandanapu	whichcat	Back-End Lead

History Table

Version	Date	Description
M1 v1	09/29/2026	Milestone 1 submitted for review
M2 v1	10/20/2026	Milestone 2 submitted for review
M3	11/03/2026	Project Status and UI Review

1. UI and Functionality Feedback (P1 Features Only)

1.1 Student Demonstration

During our M3 demo, we demonstrated the main P1 features of Nautilus, including AI-powered PRD creation, document editing and sharing, team management, review and approval workflows, notifications, and the AI chatbot. We showed how these features work together to support the PRD lifecycle.

1.2 UI and Usability Improvements Since M2

Following the M2 feedback, we expanded Nautilus beyond initial PRD generation. Users can now share documents, manage team access, review PRDs, request changes, and receive notifications. These additions help users continue working on a PRD after creating the first draft.

1.3 Instructor's Comments on the M3 Demo

During the M3 demo, the professor suggested exploring other AI tools, such as Codex, to address the limitations of Gemini's free API. She also recommended looking into MCP integration so Nautilus could connect with other tools. Her questions about the approval workflow and PRD documentation focused on understanding how the system works rather than requesting specific changes.

1.4 Team's Action Plan for M4

During M4, we will first complete the remaining work from M2: the Revision History screen, adding an entire team to a PRD, and the PRD Details Page. Version data is already stored, but the history screen remains to be built. Individual sharing is available; adding a whole team is the remaining step. The details page will bring PRD information, members, status, and approval progress together.

Based on the professor's feedback, we will look into MCP integration and explore how it could support Nautilus. We will also evaluate alternatives to Gemini to address its free API limits.

As a team, we plan to research and compare different AI models to determine which option best fits Nautilus's needs. Currently, we have not established a fixed budget or usage cap for AI services. However, through team discussions and mutual agreement, we will evaluate models based on their performance, reliability, API limitations, cost, and ability to support our existing features. Our goal is to select an AI model that provides consistent, high-quality responses while remaining practical and sustainable for the project.

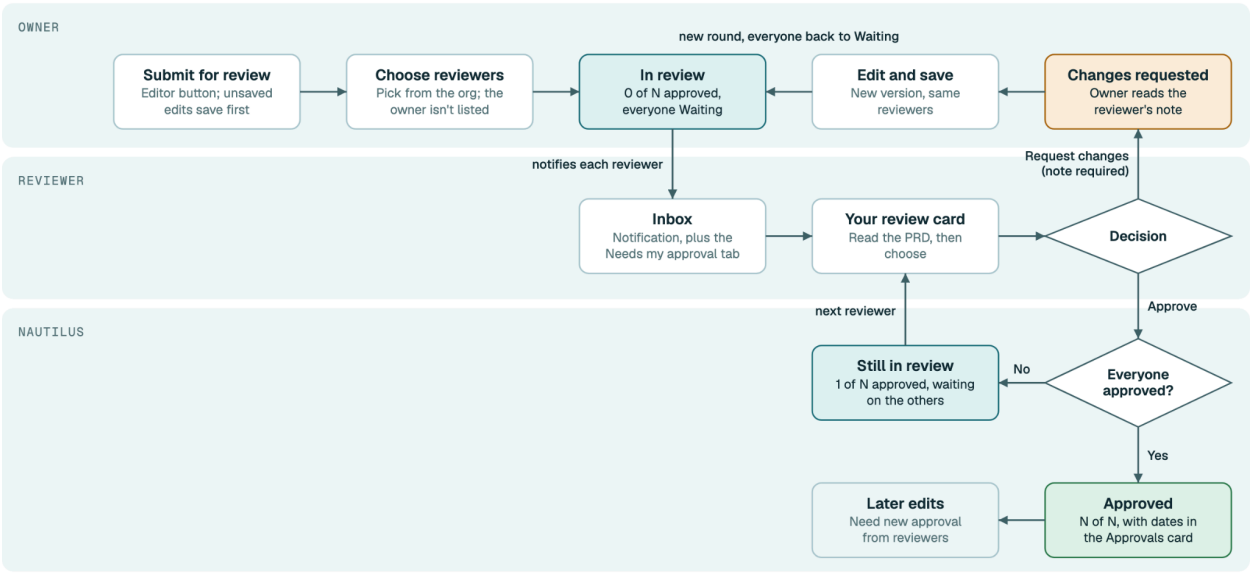
Planned M4 Work	Responsible Members
Complete the Revision History screen	Aksh, Marco, Vineela,Philip,
Add an entire team to a PRD in one action	Aksh, Marco, Vineela,Philip,
Complete the PRD Details Page	Aksh, Marco, Vineela,Leman
Explore MCP integration	Marco, Vineela, Om, Giovannie
Compare AI models based on performance, pricing, and usage limits	Om, Giovannie, Marco, Vineela

1.5 Remaining Features for M5

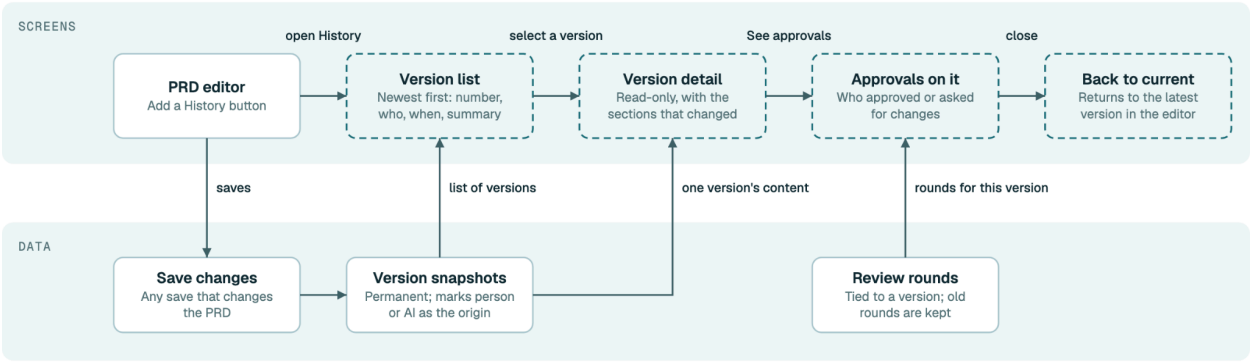
By M5, we will complete four additional features: @Mentions, AI-Suggested Teams, AI-Suggested Solutions, and Detailed Progress Reports and Status Roadmap. Section 2 describes these committed features.

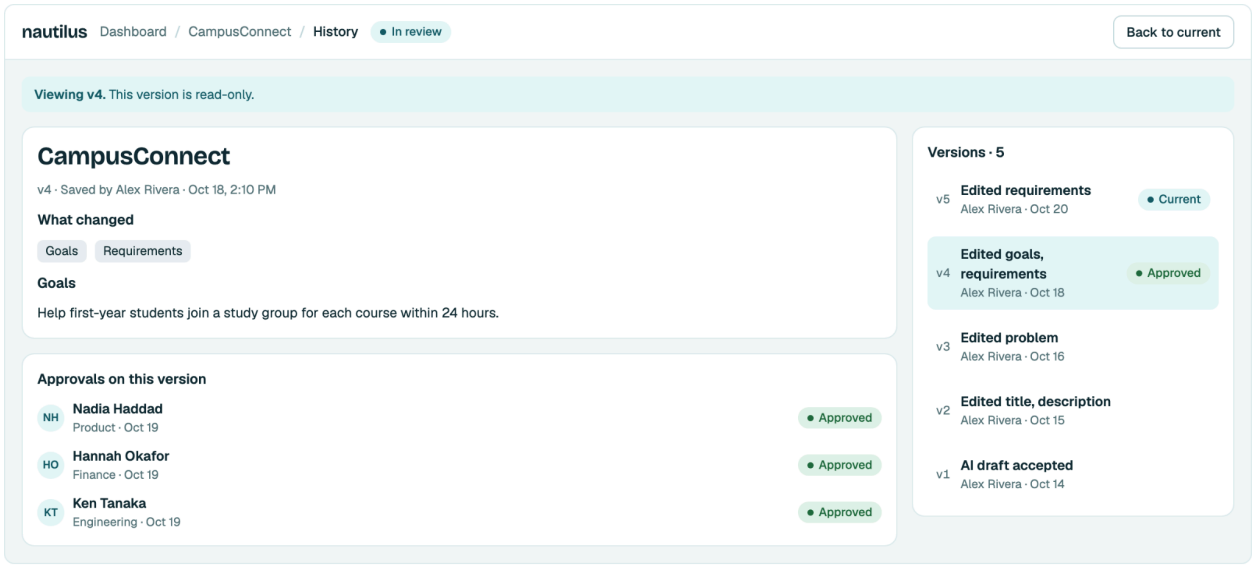
1.6 UX Flows

1. Approval_Workflow

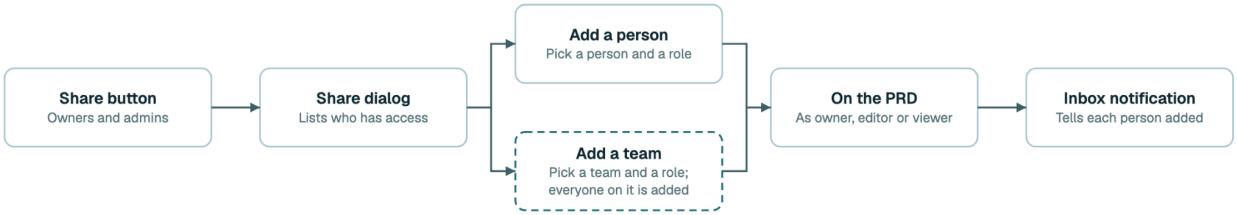


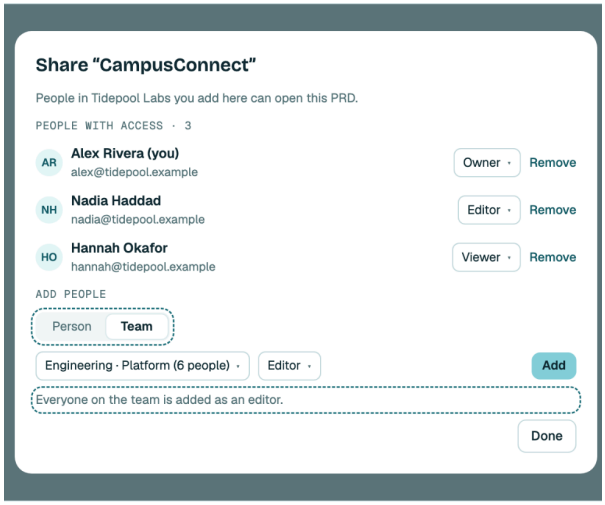
2. Revision History



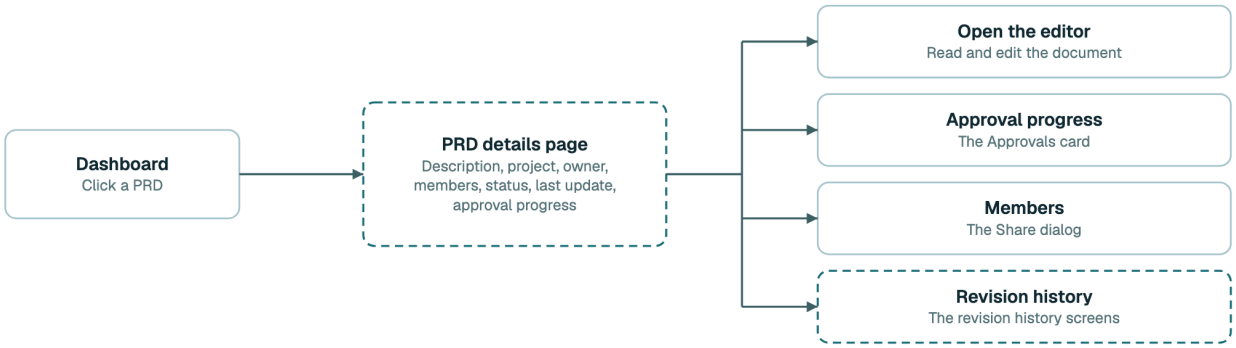


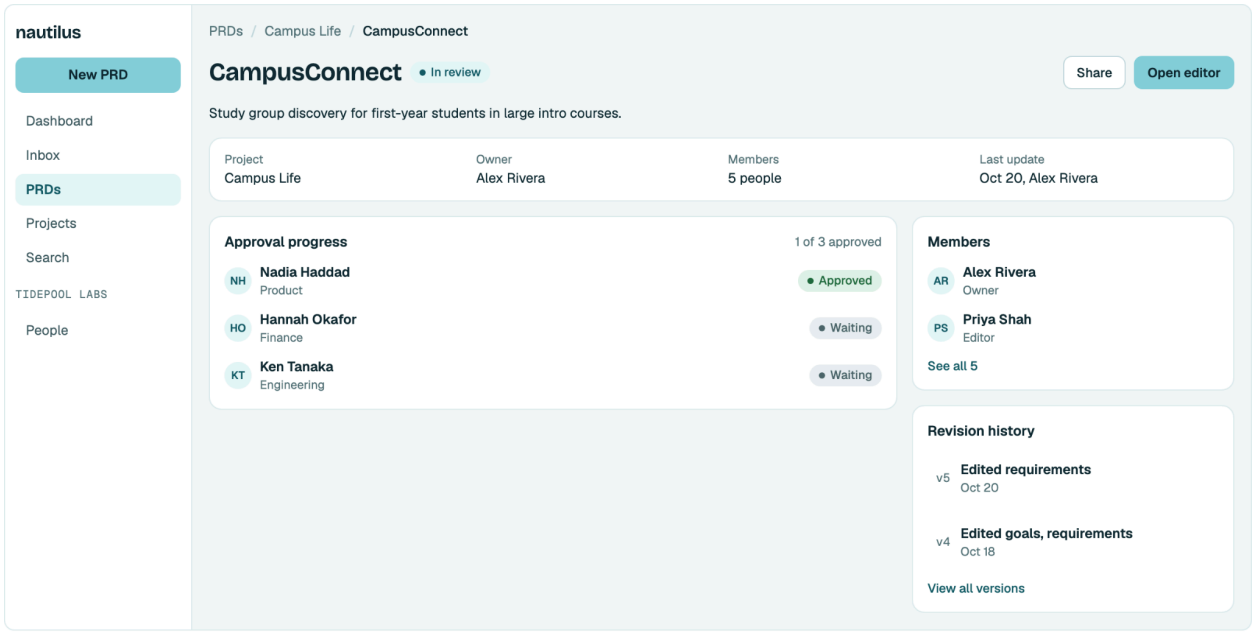
3. Add whole Team flow



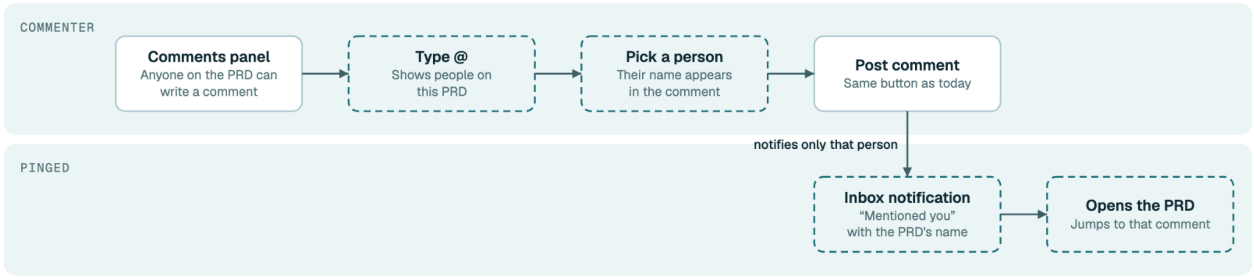


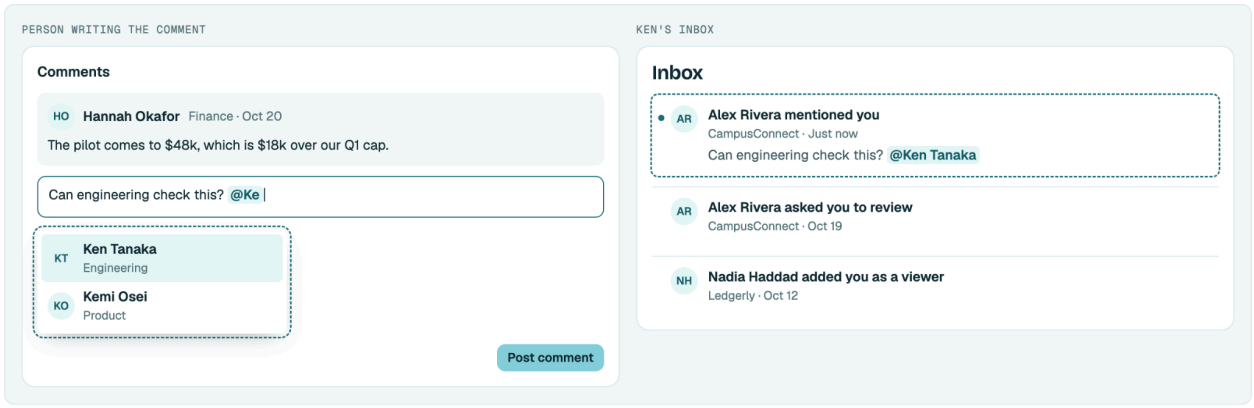
4. PRD Details flow





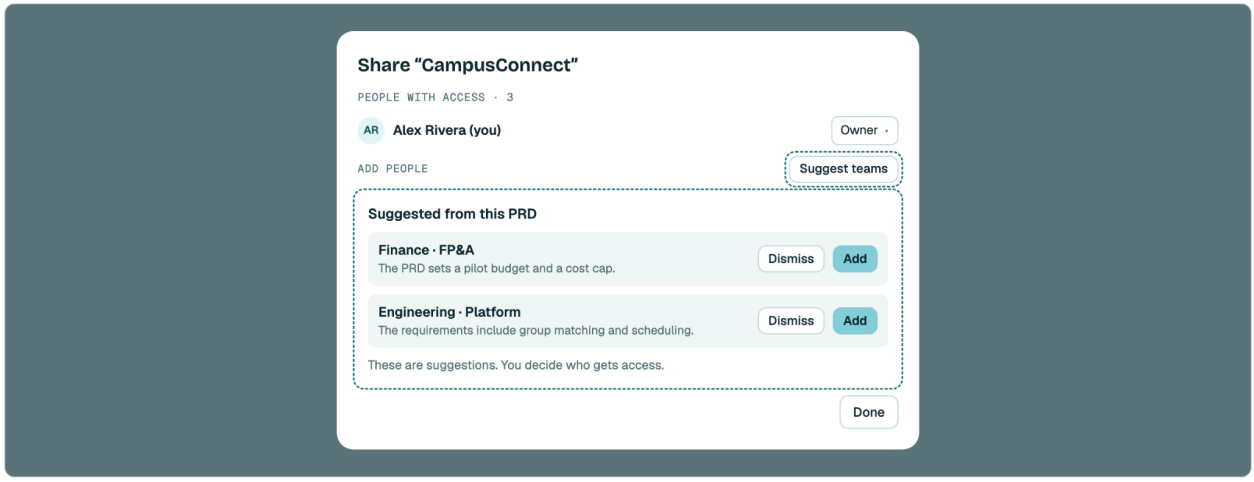
5. Mentions



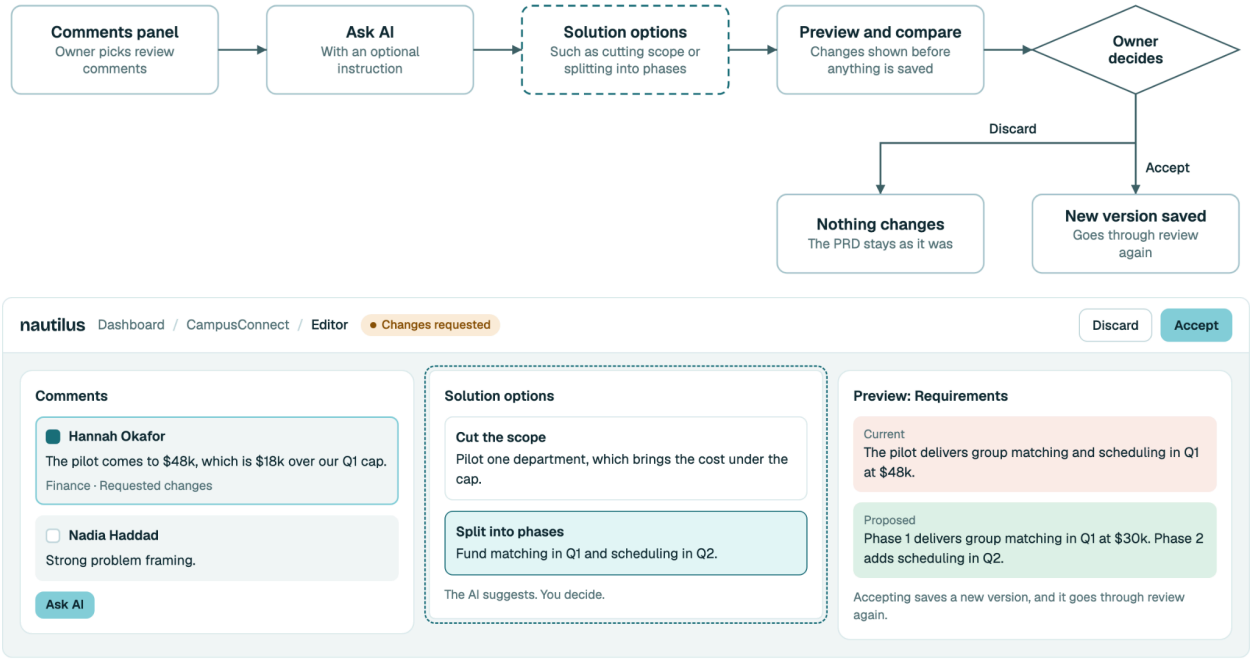


6. AI_suggested_Team

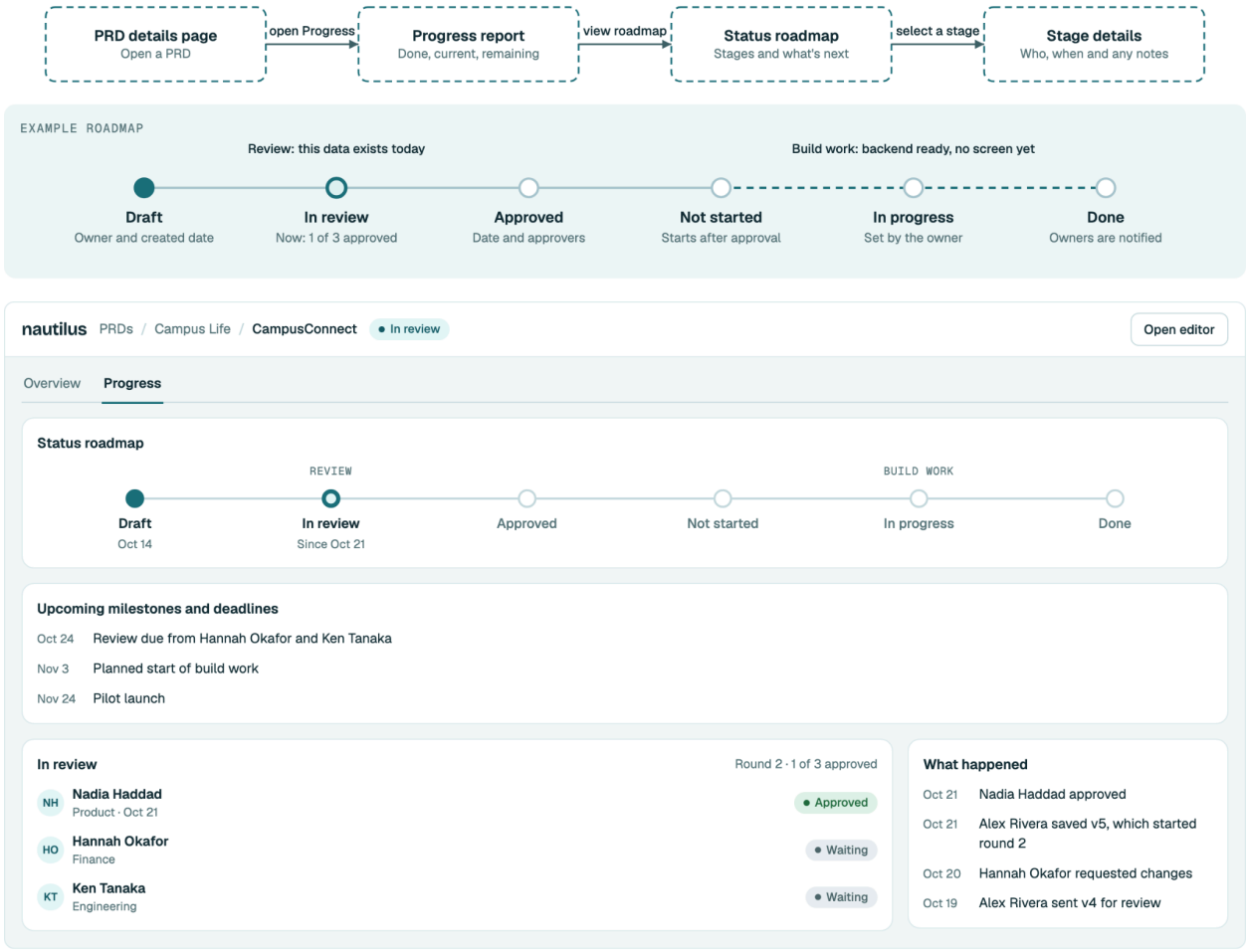




7. AI_suggested_solutions



8. Progress_roadmap



2. List of P1 Features Committed for M5 Delivery

Our team commits to completing the following four P1 features by M5.

Feature	Responsible Members	Target
@Mentions	Aksh, Marco, Vineela,Philip,	M5
AI-Suggested Teams	Om, Giovannie, Marco, Vineela	M5
AI-Suggested Solutions	Om, Aksh, Marco, Vineela	M5
Detailed Progress Reports and Status Roadmap	Aksh, Marco, Vineela,Leman	M5

2.1 @Mentions

Users will be able to mention team members using the @ symbol in PRD discussions. The mentioned member will receive a notification, helping them identify discussions that need their attention. For example, a user could write, “Hey @Aksh, can you check the login requirements?” This feature will support communication directly within Nautilus.

2.2 AI-Suggested Teams

AI will analyze a PRD and recommend which teams should contribute to the project. For example, a mobile banking PRD might require frontend, backend, and security teams. These recommendations will help users identify the teams needed for their project. The user will make the final decision about team assignments.

2.3 AI-Suggested Solutions

AI will suggest possible solutions to problems described in a PRD and its review feedback. The existing workflow provides one proposal for selected review comments. For M5, we will expand this to offer multiple solution options that users can preview and compare before accepting changes. Users will decide which suggestions to include.

2.4 Detailed Progress Reports and Status Roadmap

Users will be able to view completed work, current activities, and remaining tasks. Progress reports will show project stages, dates, and responsible people. The status roadmap will show review and work stages, including Draft, In Review, Approved, Not Started, In Progress, and Done. It will also display upcoming milestones and deadlines to help team members plan their next steps.

3. Architecture

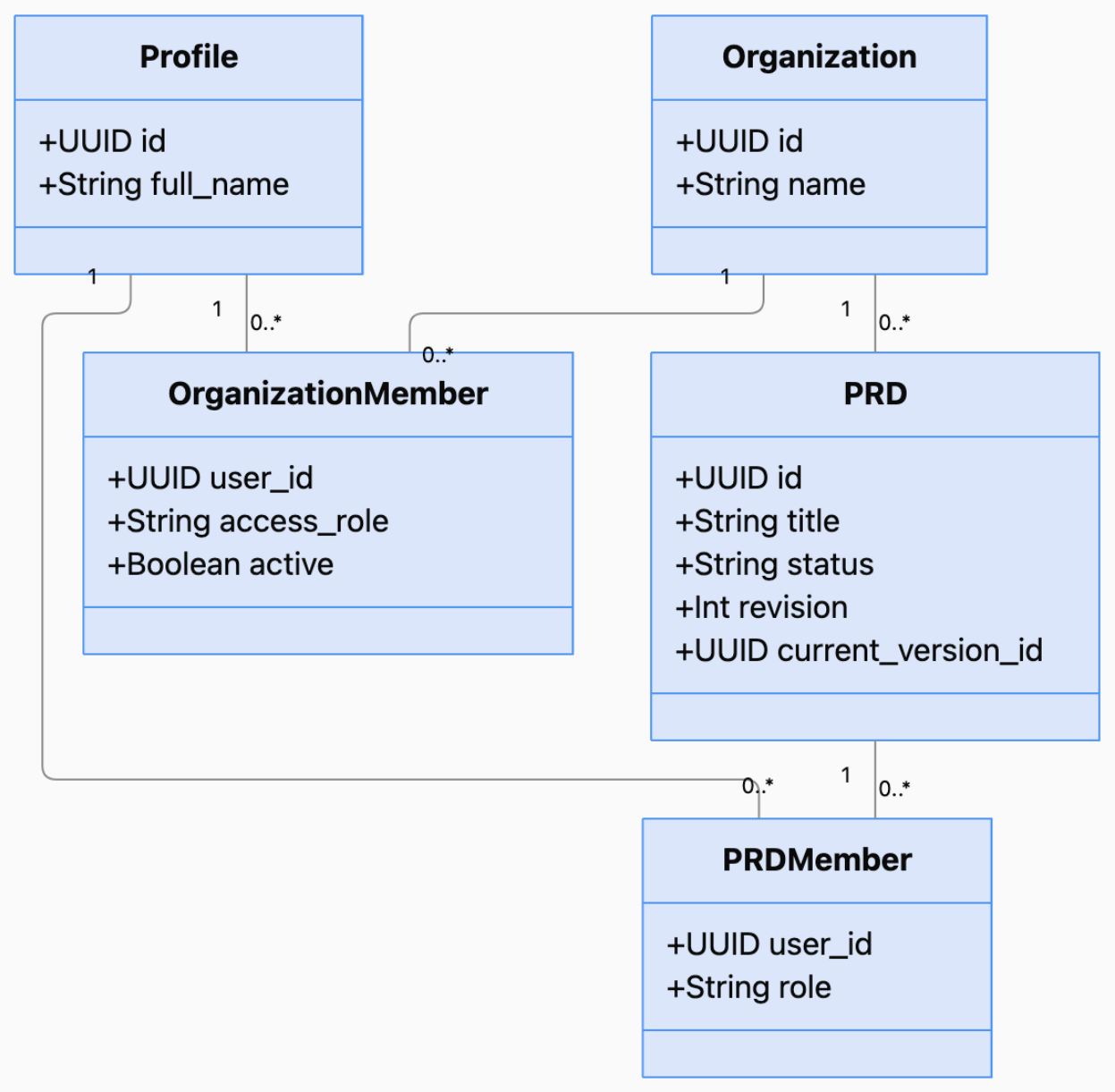
Nautilus uses a React frontend, a FastAPI backend, Supabase for authentication and data storage, and Gemini for AI functionality. The frontend communicates with Supabase Auth for sign-in and registration and calls the backend API for PRD operations. The backend validates requests, checks access, and coordinates database and AI operations.

3.1 High-Level UML Class Diagrams

These diagrams update the M2 domain model to reflect the M3 database structure. They show the main entities, selected attributes, and relationships. Entity names represent domain concepts; they do not imply that each entity has a corresponding Python class.

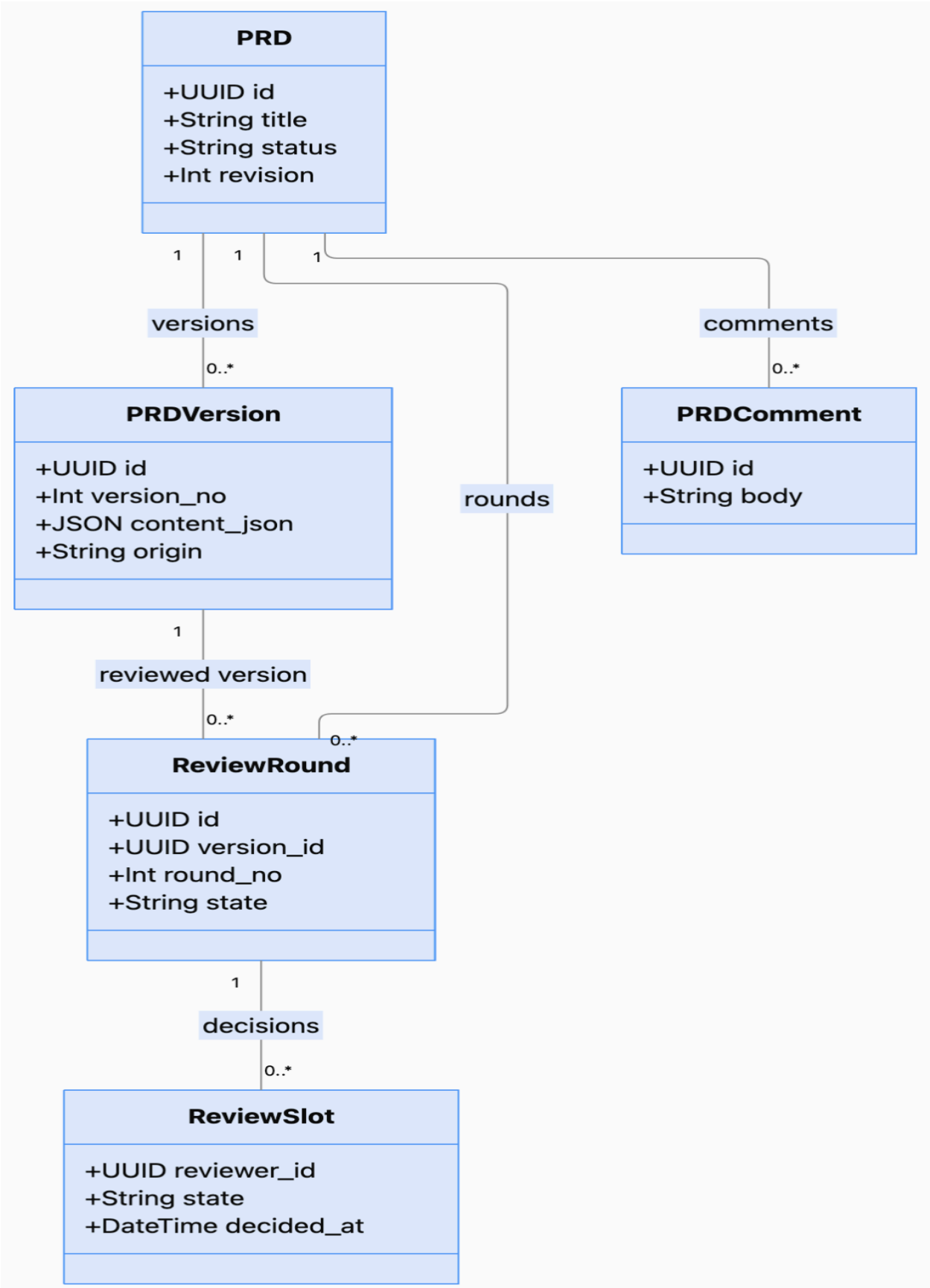
Organization and PRD Access

Organization membership determines organization access. PRD membership assigns document-specific roles such as owner, editor, or viewer. The PRD's creator is recorded separately from its membership roles.


Profile
+UUID id
+String full_name
Organization
+UUID id
+String name
OrganizationMember
+UUID user_id
+String access_role
+Boolean active
PRD
+UUID id
+String title
+String status
+Int revision
+UUID current_version_id
PRDMember
+UUID user_id
+String role
1
0..*
1
0..*
1
0..*
1
0..*
1
0..*

PRD Versions and Review Workflow

Each review round refers to a saved PRD version. Review slots record individual reviewer decisions. Version history and review history allow the application to retain earlier content and approval records.


PRD
+UUID id
+String title
+String status
+Int revision
1
1
1
versions
comments
0..*
0..*
PRDVersion
+UUID id
+Int version_no
+JSON content_json
+String origin
PRDComment
+UUID id
+String body
rounds
1
reviewed version
0..*
0..*
ReviewRound
+UUID id
+UUID version_id
+Int round_no
+String state
1
decisions
0..*
ReviewSlot
+UUID reviewer_id
+String state
+DateTime decided_at

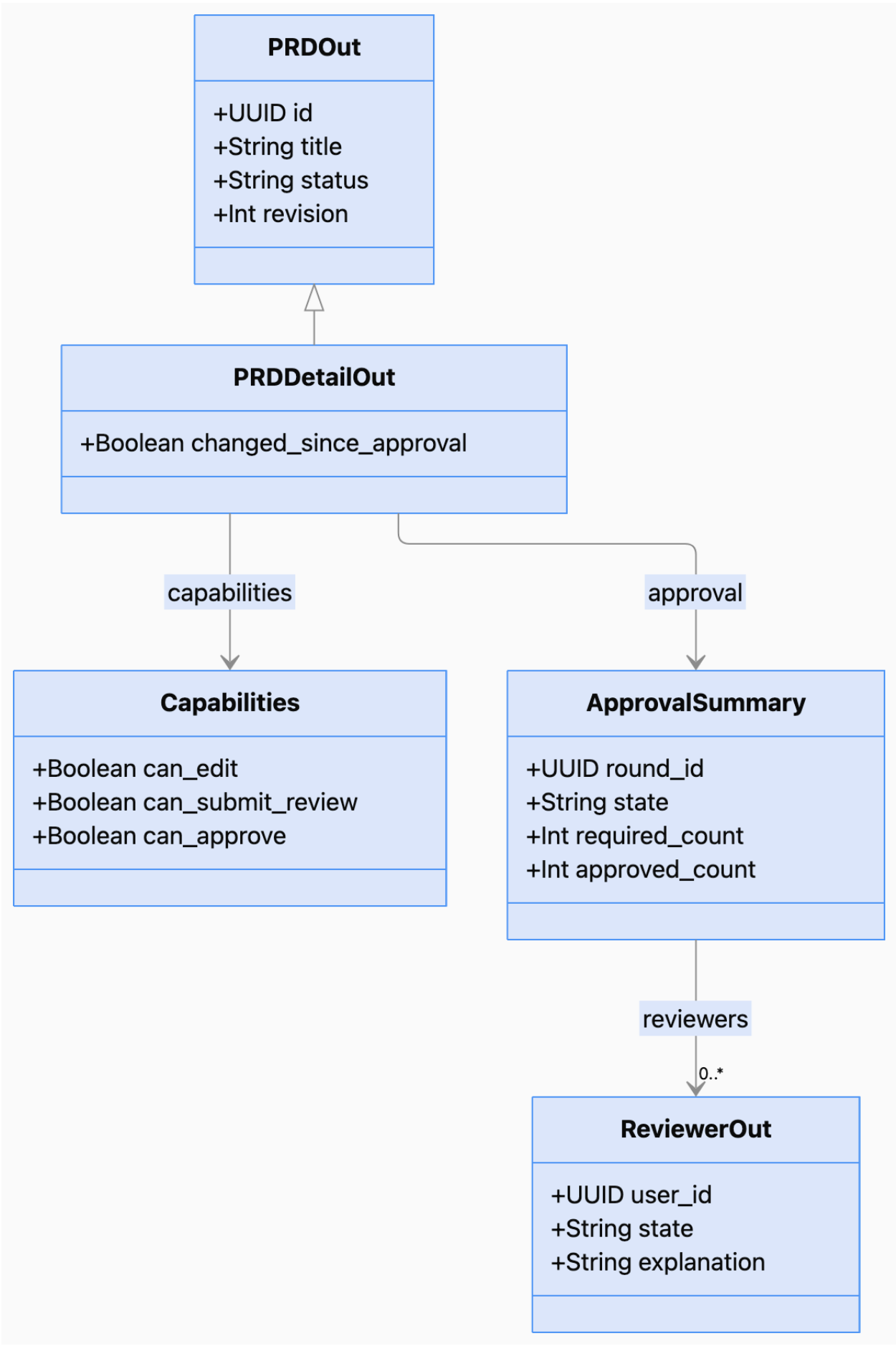
AI Generation Implementation Classes

The AI generation workflow combines the PRD's intake answers and saved context in GenerationInput. PreviewState records the revision and input fingerprint used to detect changes during generation. The generation function returns a validated GenerationProposal, which the user can review before accepting it.


GenerationProposal
+String title
+String description
+dict content_json
+String content_text
+String model
PreviewState
+Int revision
+String fingerprint
+GenerationInput inputs
inputs
GenerationInput
+String title
+String description
+Mapping answers
+SavedContext context
context
SavedContext
+tuple decisions
+tuple constraints
+tuple open_questions
+String running_summary
+from_row(row) : SavedContext

PRD Details Implementation Classes

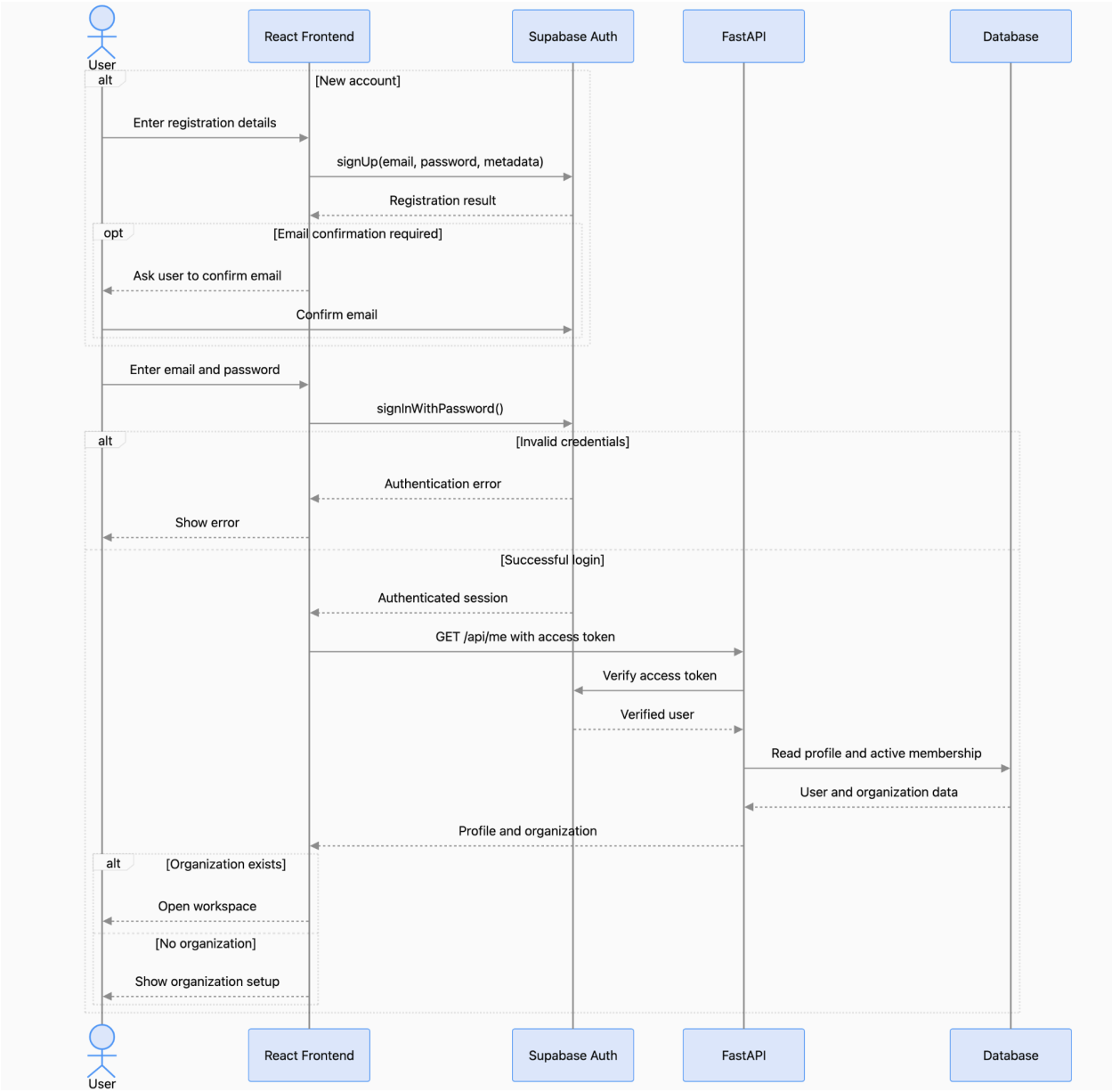
PRDDetailOut inherits the common PRD fields from PRDOut and includes access capabilities and approval information. ApprovalSummary contains the current review round's progress and individual reviewer decisions.


PRDOut
+UUID id
+String title
+String status
+Int revision
PRDDetailOut
+Boolean changed_since_approval
capabilities
approval
Capabilities
+Boolean can_edit
+Boolean can_submit_review
+Boolean can_approve
ApprovalSummary
+UUID round_id
+String state
+Int required_count
+Int approved_count
reviewers
0..*
ReviewerOut
+UUID user_id
+String state
+String explanation

3.2 High-Level Sequence Diagrams

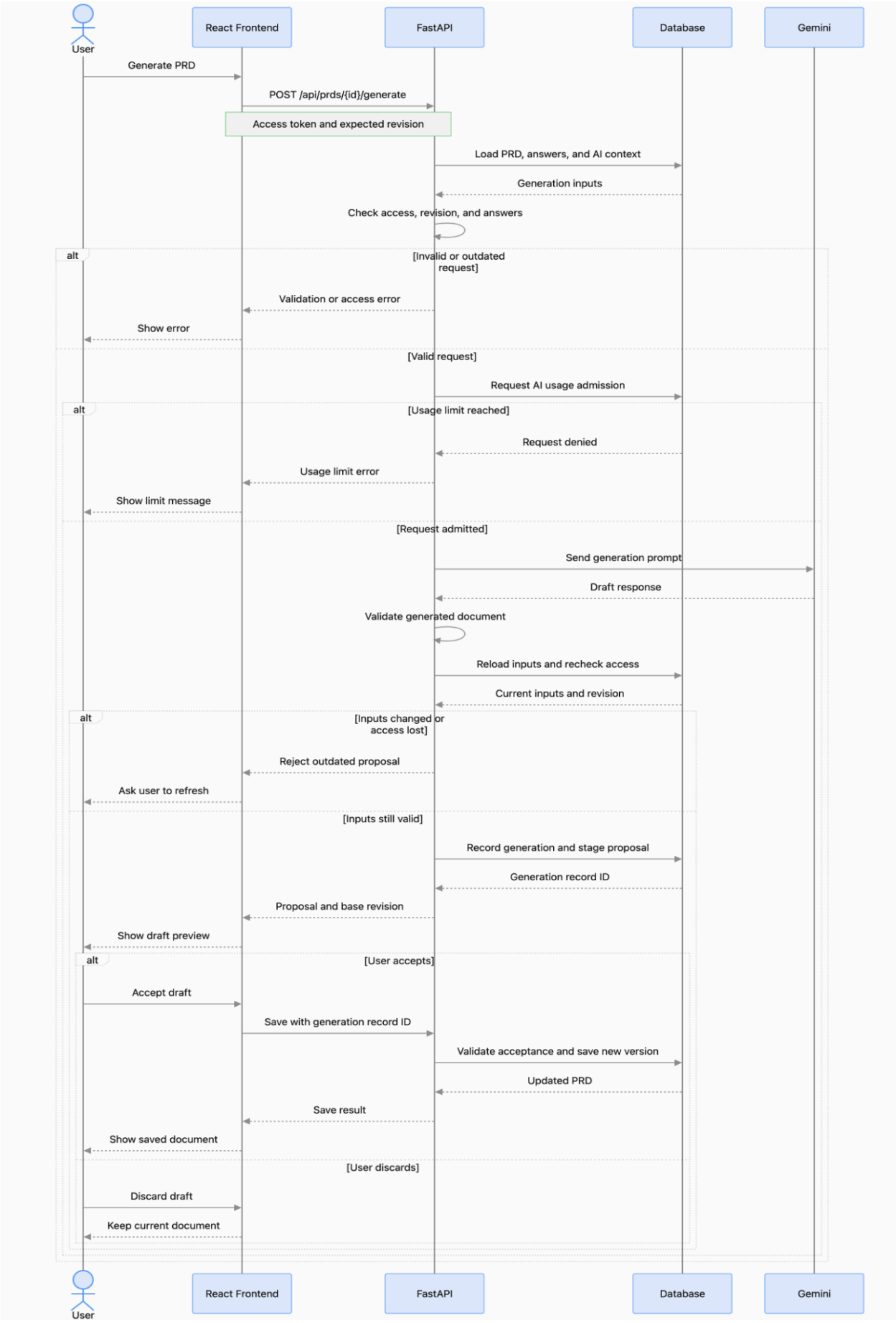
3.2.1 User Registration and Login

The frontend uses Supabase Auth for registration and login. After authentication, it requests the user's profile and organization information from FastAPI.



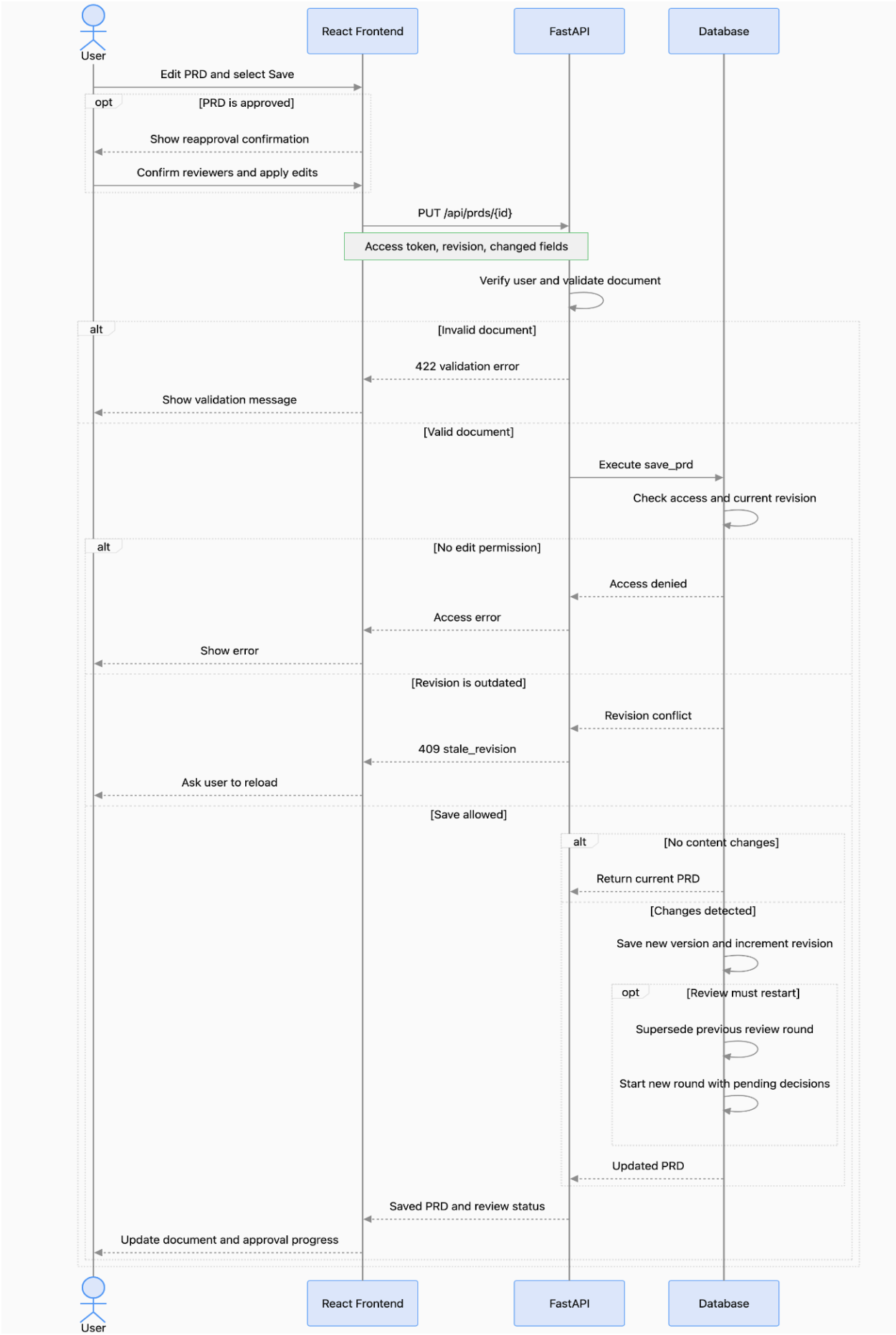
3.2.2 AI-Powered PRD Creation

The backend loads the PRD's saved intake answers and AI context, checks permissions and revision, and requests a draft from Gemini. The draft is shown as a preview. It becomes saved PRD content only after the user accepts it.


User
React Frontend
FastAPI
Database
Gemini
Generate PRD
POST /api/prds/{id}/generate
Access token and expected revision
Load PRD, answers, and AI context
Generation inputs
Check access, revision, and answers
alt
[Invalid or outdated request]
Validation or access error
Show error
[Valid request]
Request AI usage admission
alt
[Usage limit reached]
Request denied
Usage limit error
Show limit message
[Request admitted]
Send generation prompt
Draft response
Validate generated document
Reload inputs and recheck access
Current inputs and revision
alt
[Inputs changed or access lost]
Reject outdated proposal
Ask user to refresh
[Inputs still valid]
Record generation and stage proposal
Generation record ID
Proposal and base revision
Show draft preview
alt
[User accepts]
Accept draft
Save with generation record ID
Validate acceptance and save new version
Updated PRD
Save result
Show saved document
[User discards]
Discard draft
Keep current document
User
React Frontend
FastAPI
Database
Gemini

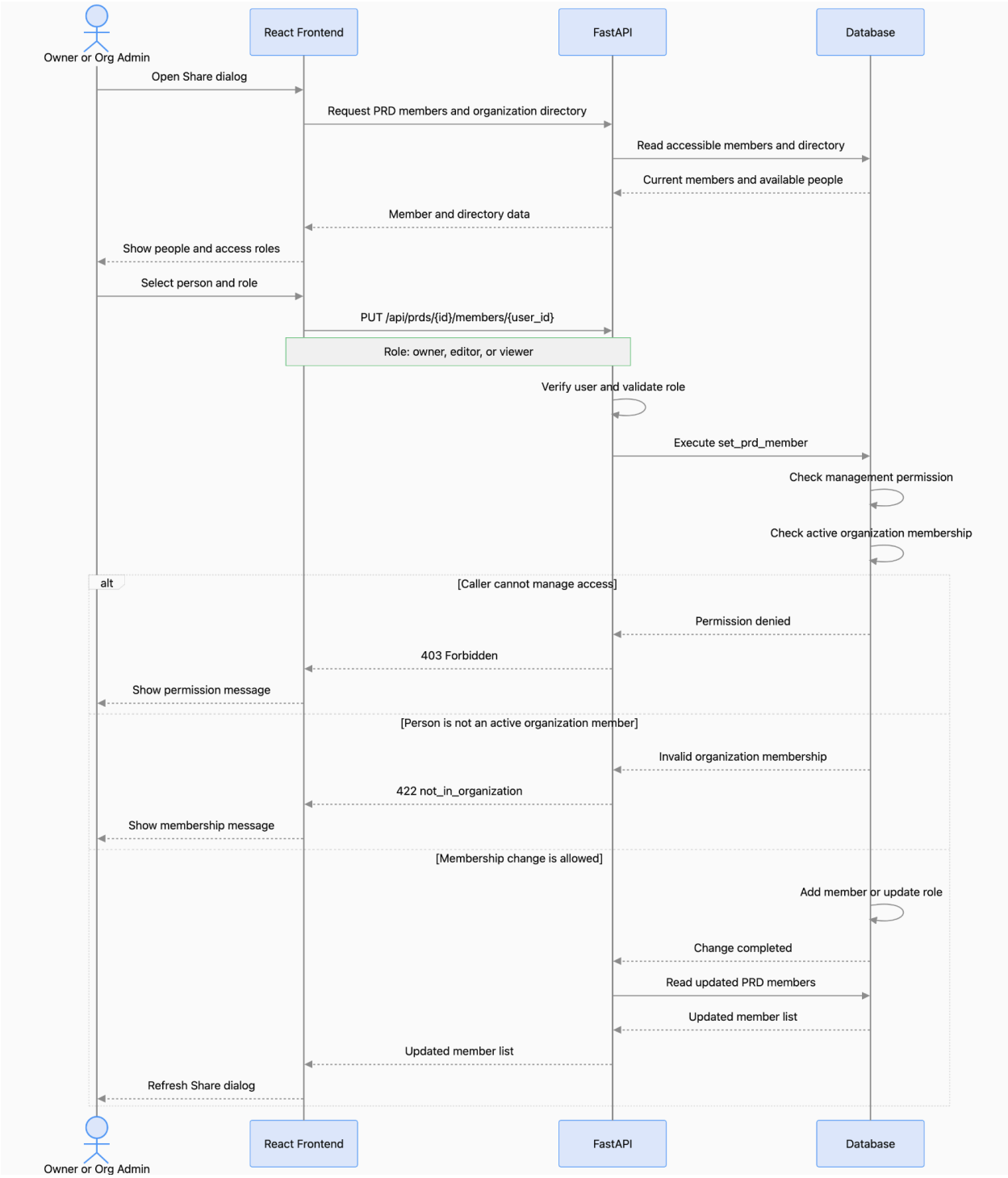
3.2.3 PRD Editing and Saving

Users save changes with the revision they loaded. The backend validates the document, and the database checks permissions and revision before saving. Editing an approved PRD requires the owner to confirm reviewers for a new review round.


User
React Frontend
FastAPI
Database
Edit PRD and select Save
opt
[PRD is approved]
Show reapproval confirmation
Confirm reviewers and apply edits
PUT /api/prds/{id}
Access token, revision, changed fields
Verify user and validate document
alt
[Invalid document]
422 validation error
Show validation message
[Valid document]
Execute save_prd
Check access and current revision
alt
[No edit permission]
Access denied
Access error
Show error
[Revision is outdated]
Revision conflict
409 stale_revision
Ask user to reload
[Save allowed]
alt
[No content changes]
Return current PRD
[Changes detected]
Save new version and increment revision
opt
[Review must restart]
Supersede previous review round
Start new round with pending decisions
Updated PRD
Saved PRD and review status
Update document and approval progress
User
React Frontend
FastAPI
Database

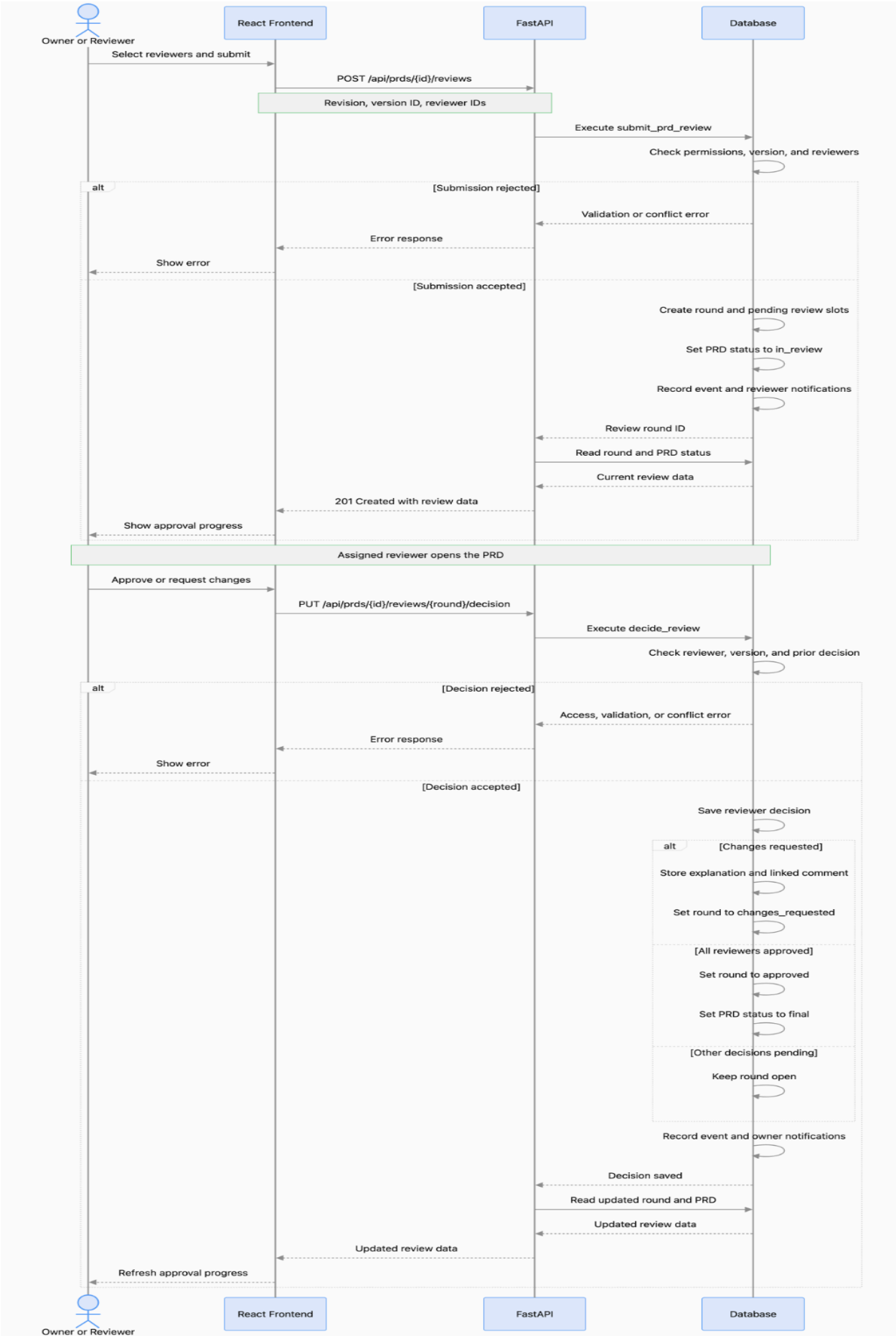
3.2.4 Document Sharing and Access Management

PRD owners and organization administrators can add people from the organization and assign owner, editor, or viewer access. The database checks permissions and membership before applying changes.


Owner or Org Admin
React Frontend
FastAPI
Database
Open Share dialog
Request PRD members and organization directory
Read accessible members and directory
Current members and available people
Member and directory data
Show people and access roles
Select person and role
PUT /api/prds/{id}/members/{user_id}
Role: owner, editor, or viewer
Verify user and validate role
Execute set_prd_member
Check management permission
Check active organization membership
alt
[Caller cannot manage access]
Permission denied
403 Forbidden
Show permission message
[Person is not an active organization member]
Invalid organization membership
422 not_in_organization
Show membership message
[Membership change is allowed]
Add member or update role
Change completed
Read updated PRD members
Updated member list
Updated member list
Refresh Share dialog
Owner or Org Admin
React Frontend
FastAPI
Database

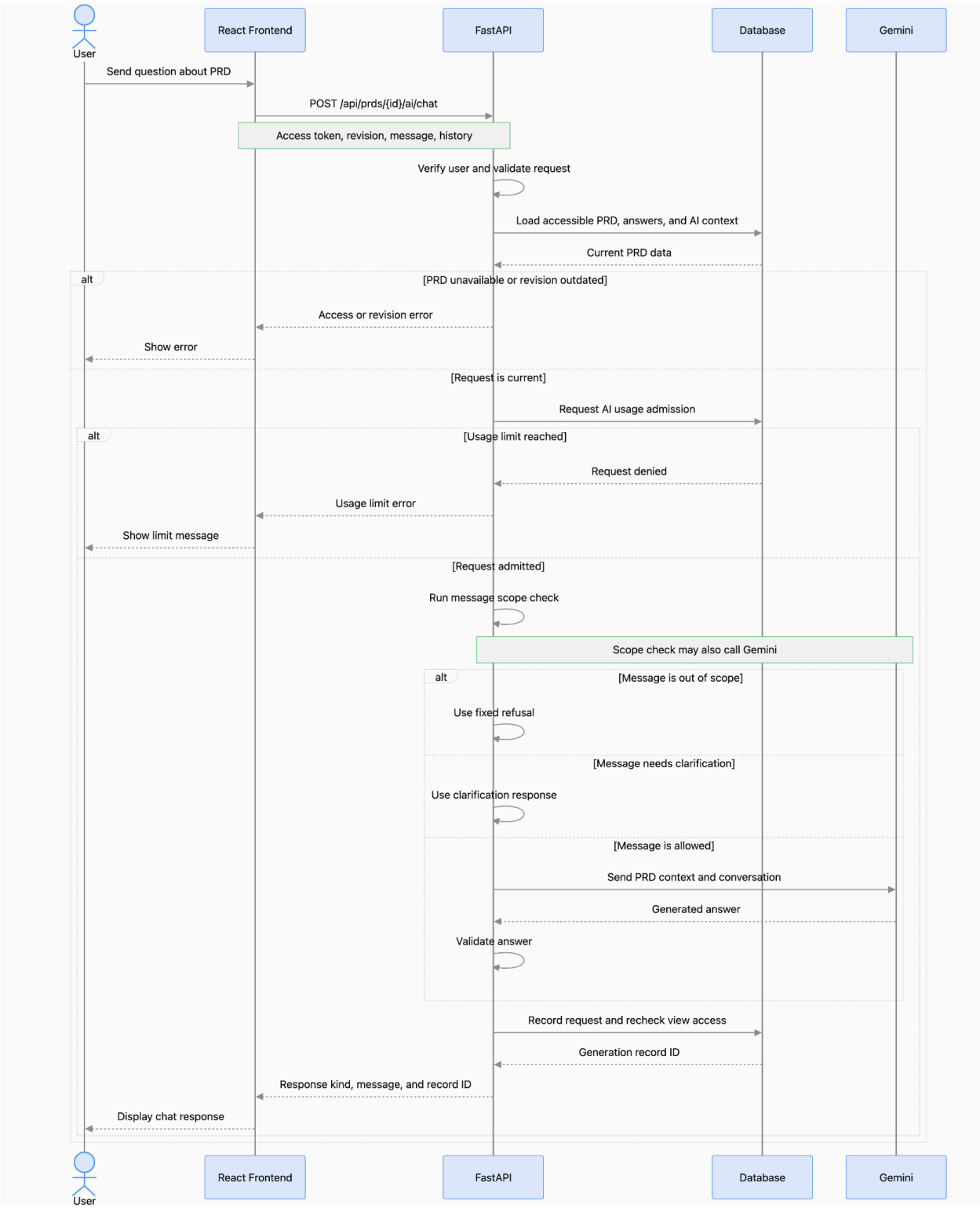
3.2.5 Review and Approval Workflow

Owners or organization administrators submit a saved PRD version for review. Each assigned reviewer approves it or requests changes. The PRD becomes final when every reviewer approves.


Owner or Reviewer
React Frontend
FastAPI
Database
Select reviewers and submit
POST /api/prds/{id}/reviews
Revision, version ID, reviewer IDs
Execute submit_prd_review
Check permissions, version, and reviewers
alt
[Submission rejected]
Validation or conflict error
Error response
Show error
[Submission accepted]
Create round and pending review slots
Set PRD status to in_review
Record event and reviewer notifications
Review round ID
Read round and PRD status
Current review data
201 Created with review data
Show approval progress
Assigned reviewer opens the PRD
Approve or request changes
PUT /api/prds/{id}/reviews/{round}/decision
Execute decide_review
Check reviewer, version, and prior decision
alt
[Decision rejected]
Access, validation, or conflict error
Error response
Show error
[Decision accepted]
Save reviewer decision
alt
[Changes requested]
Store explanation and linked comment
Set round to changes_requested
[All reviewers approved]
Set round to approved
Set PRD status to final
[Other decisions pending]
Keep round open
Record event and owner notifications
Decision saved
Read updated round and PRD
Updated review data
Updated review data
Refresh approval progress
Owner or Reviewer
React Frontend
FastAPI
Database

3.2.6 AI Chatbot Interaction

Users can ask questions about a PRD they can access. The backend checks the document revision, AI usage limits, and message scope before generating an answer. Chat responses do not change the PRD.


User
React Frontend
FastAPI
Database
Gemini
Send question about PRD
POST /api/prds/{id}/ai/chat
Access token, revision, message, history
Verify user and validate request
Load accessible PRD, answers, and AI context
Current PRD data
alt
[PRD unavailable or revision outdated]
Access or revision error
Show error
[Request is current]
Request AI usage admission
alt
[Usage limit reached]
Request denied
Usage limit error
Show limit message
[Request admitted]
Run message scope check
Scope check may also call Gemini
alt
[Message is out of scope]
Use fixed refusal
[Message needs clarification]
Use clarification response
[Message is allowed]
Send PRD context and conversation
Generated answer
Validate answer
Record request and recheck view access
Generation record ID
Response kind, message, and record ID
Display chat response
User
React Frontend
FastAPI
Database
Gemini

4. Project Status

4.1 Current Development Status

During M3, we demonstrated AI-powered PRD creation, document editing and sharing, team management, review and approval workflows, notifications, and the AI chatbot.

For M4, our first priority is to complete the Revision History screen, whole-team sharing, and the PRD Details Page carried over from M2. We will also explore MCP integration and compare AI models based on performance, pricing, and usage limits. For M5, we will complete the four committed features described in Section 2.

4.2 Actual Key Risks

Owner: Om, Leman **Support:** All team members

Category	Risk	How We Will Resolve It	Owner
Technical / AI	Gemini's free API limits have restricted repeated testing of PRD generation and AI interactions. This reduces the number of tests we can complete in one session.	We will prioritize essential test cases, coordinate AI testing, and evaluate alternative AI tools based on the professor's feedback.	Om, Giovannie, Marco, Vineela
Technical / Email	Supabase's email limits restrict repeated testing of password reset and other email-dependent flows.	We will schedule email-dependent tests carefully and use password or Google sign-in during demonstrations. We will verify the password reset flow separately.	Aksh, Marco, Vineela,Leman
Schedule	The Revision History screen, whole-team sharing, and PRD Details Page remain unfinished from M2. Completing these tasks alongside MCP research and AI model evaluation adds pressure to the M4 schedule.	We will prioritize the unfinished M2 work, divide tasks into smaller steps, and track progress and blockers during meetings.	Om,Philip, Leman,

Category	Risk	How We Will Resolve It	Owner
Teamwork / Integration	Frontend and backend work depend on shared decisions about permissions, workflows, and API responses. The team identified a need to clarify these details so both sides could develop consistently.	We will maintain the shared implementation reference, communicate API changes on Discord, and test connected workflows as changes are integrated.	Aksh, Marco, Vineela,Philip,

4.3 Team Collaboration and Development Workflow

Our team uses Discord to share progress, discuss blockers, and coordinate tasks. Meeting summaries are stored in the repository under milestones/m3/meetings, so decisions and next steps are documented alongside the code.

We develop changes on feature branches and submit pull requests for review before merging into develop. Reviewed changes are then prepared for release to main. Code reviews focus on functionality, integration, and specific issues that need attention.

Every pull request into develop or main runs automated checks through GitHub Actions: backend unit tests, backend integration tests against a local Supabase stack, and frontend lint and build. The ruleset protecting develop and main requires a pull request, one approval, resolved review conversations and passing checks before merging, dismisses approvals when new commits are pushed, and blocks force pushes and branch deletion. CODEOWNERS requests reviewers automatically by folder, and a PR template standardizes descriptions. Main is updated only through release branches; the M3 release went through release/m3 into main.